DOCUMENTAÇÃO TÉCNICA & ARQUITETURA DE SOFTWARE

Projeto LuArtes: Pipeline de Análise de Vendas

PROJETO	LINGUAGEM / SO	AMBIENTE	STATUS
Análise de Vendas LuArtes	**Python 3.11.2 (Linux)**	**VS Code + Anaconda**	**Concluído com Sucesso**

1. Resumo Executivo & Objetivos

O projeto **LuArtes** consistiu na concepção e implementação de uma solução completa de engenharia e análise de dados para o e-commerce corporativo de produtos personalizados LuArtes. O sistema foi projetado em Python para simular a ingestão de transações comerciais dinâmicas, processar regras de negócio, consolidar indicadores de desempenho gerencial (KPIs), gerar visualizações gráficas corporativas e exportar relatórios padronizados compatíveis com ferramentas de planilha como LibreOffice Calc e Microsoft Excel.

2. Stack Tecnológica e Especificações de Ambiente

A infraestrutura de execução foi estabelecida em ambiente Linux (distribuição corporativa), gerenciada via terminal e integrando dependências científicas de ponta.

Tecnologia / Biblioteca	Versão Utilizada	Função no Ecossistema LuArtes
Python	`v3.11.2`	Linguagem base para construção do pipeline de dados e funções gerenciais.
Pandas	`v2.x+`	Modelagem de DataFrames, tratamento, junções e agrupamento (`groupby`).
NumPy	`v1.26+`	Operações vetoriais eficientes para suporte às estruturas de dados.
Matplotlib	`v3.8+`	Motor de renderização gráfica bidimensional e controle de figuras.
Seaborn	`v0.13+`	Camada de estilização estatística para geração do gráfico de barras corporativo.
LibreOffice Calc / VS Code	`Última Estável`	Edição de código, inspeção de CSVs e validação de relatórios exportados.

3. Engenharia de Dados & Regras de Negócio

O pipeline gera 100 registros transacionais aleatórios dentro de uma janela temporal dinâmica de 30 dias. A tabela catálogo fixa os preços unitários por categoria, permitindo o cruzamento de dados (`merge`) e cálculo do faturamento.

Catálogo e Precificação Mapeada

• **Caneca Porcelana:** R$ 45,00 | • **Caneca Polímero Infantil:** R$ 30,00

4. Resolução de Desafios Técnicos no Linux

Durante o desenvolvimento da etapa gráfica no ambiente Linux (servidor de exibição Wayland/X11 com interface GNOME), a tentativa inicial de exibição interativa via `plt.show()` resultou num erro de plataforma Qt/XCB (`qt.qpa.plugin: Could not load Qt platform plugin`), abortando o processo.

Engenharia da Solução Gráfica (Backend 'Agg')

A falha foi sanada definitivamente forçando o Matplotlib a operar em modo não-interativo através do comando `matplotlib.use('Agg')` declarado no topo do código, antes da importação do `pyplot`. Isso redirecionou a renderização diretamente para o disco como imagem estática PNG (`plt.savefig`) com densidade de 300 DPI.

5. Resultados Consolidados do Relatório Gerencial

O processamento do relatório final produziu resultados financeiros consistentes com a simulação transacional de volume elevado do e-commerce.

R$ 13.895,00	308 Unidades
FATURAMENTO TOTAL PROCESSADO	VOLUME TOTAL DE ITENS VENDIDOS

6. Estrutura Final do Código Python (`analise_vendas.py`)

```
# ========================================= # PROJETO: Análise de Vendas da LuArtes # ========================================= import matplotlib matplotlib.use('Agg') # Força o Matplotlib a trabalhar apenas em modo de salvamento, sem abrir janelas import matplotlib.pyplot as plt import pandas as pd import numpy as np import seaborn as sns from datetime import datetime, timedelta import random print("Ambiente carregado e bibliotecas prontas para uso!") # Catálogo da LuArtes produtos_luartes = [ "Caneca Porcelana", "Caneca Polímero Infantil", "Caneca com Colher", "Caneca Mágica Térmica", "Caneca de Alumínio Chopp", "Garrafa squeeze" ] precos = [45.0, 30.0, 50.0, 65.0, 55.0, 30.0] print("Catálogo de produtos da LuArtes configurado com sucesso!") # ========================================= # 3. GERAÇÃO DE HISTÓRICO DE VENDAS # ========================================= # Vamos simular 100 vendas passadas da LuArtes n_vendas = 100 dados_vendas = { "Data": [datetime.now() - timedelta(days=random.randint(0, 30)) for _ in range(n_vendas)], "Produto": [random.choice(produtos_luartes) for _ in range(n_vendas)], "Quantidade": [random.randint(1, 5) for _ in range(n_vendas)] } # Criando o nosso DataFrame (tabela de dados) com Pandas df_vendas = pd.DataFrame(dados_vendas) print("\n--- Primeiras vendas geradas com sucesso ---") print(df_vendas.head()) # ========================================= # 4. ADICIONANDO PREÇOS E CALCULANDO FATURAMENTO # ========================================= # Mapeando os preços correspondentes a cada produto vendido na tabela mapeamento_precos = dict(zip(produtos_luartes, precos)) df_vendas["Preco_Unitario"] = df_vendas["Produto"].map(mapeamento_precos) # Calculando o valor total da venda (Quantidade * Preço Unitário) df_vendas["Faturamento_Total"] = df_vendas["Quantidade"] * df_vendas["Preco_Unitario"] print("\n--- Tabela Completa de Vendas da LuArtes (Primeiras Linhas) ---") print(df_vendas.head()) # ========================================= # 5. RESUMO E MÉTRICAS DE NEGÓCIO DA LUARTES # ========================================= # Faturamento total somando toda a coluna de faturamento faturamento_total = df_vendas["Faturamento_Total"].sum() # Quantidade total de itens vendidos total_itens_vendidos = df_vendas["Quantidade"].sum() # Vendas agrupadas por tipo de produto vendas_por_produto = df_vendas.groupby("Produto")[["Quantidade", "Faturamento_Total"]].sum() print(f"\n=========================================") print(f" RELATÓRIO GERENCIAL - LUARTES ") print(f"=========================================") print(f"💰 Faturamento Total: R$
```